

Hackathon 2026 Talento Tech 2

Documentación del MVP: Infraestructura Command & Control (C2)

Desarrollado por el Grupo:
Sarabhel

Organizado por:
Aligo Defensores Informáticos
Medellín, Colombia

Especificación de Requerimientos y Funcionamiento Asíncrono

Resumen

Este documento consolida el diseño conceptual, los requerimientos funcionales y no funcionales, así como la arquitectura de microservicios desarrollada para el reto de la Hackathon 2026 Aligo por el equipo Sarabhel. El sistema expone un entorno Command & Control (C2) controlado mediante flujos automatizados, una base de datos relacional expuesta vía API y un componente ejecutor remoto bajo la estrategia de sondeo periódico.

1 Contexto

Este reto es organizado por **Aligo Defensores Informáticos**, una empresa de ciberseguridad y hacking con 17 años de trayectoria nacida en la ciudad de Medellín. La empresa desarrolla actividades relacionadas con *Red Teaming*, *Blue Teaming*, *Purple Teaming*, *Green Teaming* y múltiples áreas asociadas a la seguridad informática.

A diferencia de un CTF tradicional, este evento es un **hackathon de desarrollo**: los participantes no resuelven retos prearmados, sino que construyen su propio proyecto técnico desde cero a lo largo de la competencia.

Un C2 es un concepto fundamental dentro de la ciberseguridad y toca, de manera natural, una enorme cantidad de disciplinas técnicas: redes, criptografía, arquitectura de sistemas distribuidos, diseño de protocolos, serialización, concurrencia y desarrollo de software. Por eso el reto no está limitado a personas del mundo de la ciberseguridad: cualquier rama de la tecnología puede aportar y destacar, porque construir un buen C2 combina muchas competencias a la vez.

Importante: Todo el desarrollo y las pruebas se realizan exclusivamente en un entorno de laboratorio cerrado y autorizado, provisto por la organización. El objetivo es demostrar dominio técnico y diseño de ingeniería, no operar contra sistemas reales ni de terceros.

2 Estructura C2

En ciberseguridad, un entorno *Command and Control* (C2) es una infraestructura centralizada que permite a un operador (atacante o equipo *red team*) mantener comunicación, enviar instrucciones y recibir datos de sistemas comprometidos o agentes desplegados de forma remota.

El objetivo principal de un C2 no es el ataque inicial sino la persistencia y la administración remota de una red de máquinas (“bots” o “agentes”) de manera asíncrona, sigilosa y organizada, separando la identidad y ubicación del operador de las máquinas objetivo.

El proyecto entrega una infraestructura C2 basada en la nube y construida con una arquitectura de microservicios. Consta de tres componentes principales:

2.1 La infraestructura del operador (n8n)

Funciona como el panel de control interactivo. A través de flujos automatizados y formularios web, el operador puede enviar comandos de forma amigable sin interactuar directamente con la base de datos del sistema.



Figura 1: Flujo de automatización e integración en n8n para la gestión de tareas del operador.

En la Figura 1 se puede observar el pipeline automatizado compuesto por disparadores de planificación (*Schedule Trigger*), procesamiento de consultas e instrucciones JavaScript, y el componente final de conversión HTML a PDF.

2.2 Base de datos en la nube (Supabase - Postgre)

Actúa como el núcleo de la infraestructura alojado en la nube, gestiona dos tablas a través de una API REST:

- **Agents:** Mantiene el inventario y estado de todas las máquinas infectadas, registrando su último *heartbeat* y su último resultado.
- **Tasks:** Funciona como una cola de mensajes asíncrona. Almacena las órdenes enviadas por el operador y espera a que los agentes las reclamen, ejecuten y devuelvan el resultado.

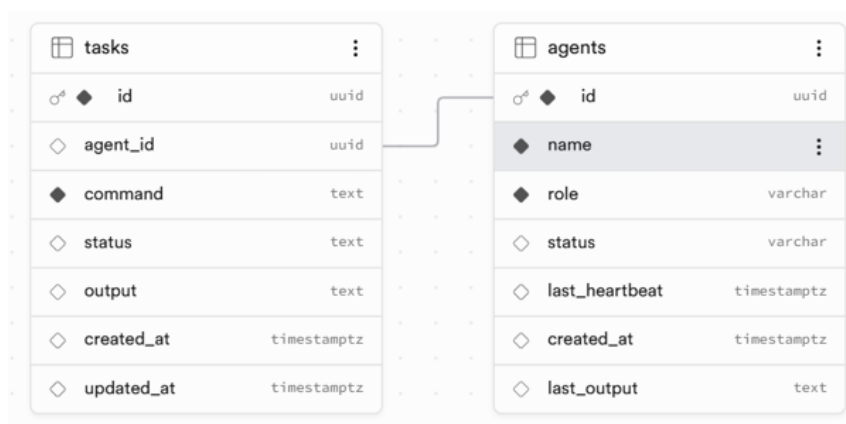


Figura 2: Modelo de Entidad-Relación entre las tablas **tasks** y **agents**.

Como se ilustra en el esquema de la Figura 2, la base de datos vincula ambas entidades mediante el campo de identidad lógica relacional.

2.3 El agente (Script)

Es el “malware” o cliente remoto que se ejecuta en la máquina objetivo. Trabaja mediante una estrategia de *Polling* (consulta periódica), enviando señales de vida y revisando la tabla de tareas. Al encontrar una orden pendiente, la ejecuta en el sistema operativo subyacente y devuelve la salida (*output*) al servidor central.

```

Run agent x
C:\Users\crist\PycharmProjects\Hackaton\.venv\Scripts\python.exe C:\Users\crist\PycharmProjects\Hackaton\agent.py
[*] Registrando agente Agente-Python-01 (executor)...
[+] Agente reconectado. Estado cambiado a 'online'.
[*] Iniciando bucle de Heartbeat y revisión de tareas (CTRL+C para salir)...
[-] Heartbeat enviado a las 08:27:01
[+] Tarea completada. Resultado y 'last_output' actualizados.
[-] Heartbeat enviado a las 08:27:07
[+] Tarea completada. Resultado y 'last_output' actualizados.

```

Figura 3: Consola de salida en tiempo real del agente ejecutando tareas en segundo plano.

La traza de ejecución expuesta en la Figura 3 evidencia el registro inicial del componente executor, seguido de los bucles iterativos de latidos periódicos y la resolución exitosa de comandos remotos.

3 Requerimientos del Reto

Los requerimientos técnicos presentados para la validación del reto son:

- Un servidor C2 que acepte la conexión de al menos un agente.
- Un agente que se conecte al servidor y ejecute comandos enviados de forma remota.
- Retorno de los resultados de esos comandos al operador.
- El canal de comunicación funcionando de extremo a extremo durante la demostración.
- Los cuatro entregables (código, documentación, video y demo) presentados al cierre.

3.1 Requerimientos Funcionales

Con base en los objetivos iniciales, se formalizan los siguientes requerimientos funcionales del sistema:

RF-01. Registro y telemetría de agentes. El sistema deberá registrar el estado de conexión y la hora del último latido (*heartbeat*) en la base de datos central cuando el script del agente inicie su ejecución en la máquina objetivo.

RF-02. Asignación de comandos. El sistema deberá registrar una nueva tarea en estado pendiente dentro de la base de datos cuando el operador envíe una instrucción a través del formulario web de n8n.

RF-03. Ejecución remota de tareas. El sistema deberá ejecutar el comando especificado a nivel del sistema operativo local cuando el agente consulte la base de datos y detecte una nueva tarea pendiente asignada a su identificador.

RF-04. Retorno de resultados. El sistema deberá actualizar el registro de la tarea y el último estado del agente con la información extraída (*output*) cuando el agente finalice exitosamente la ejecución del comando en la terminal.

RF-05. Generación de reportes de estado. El sistema deberá generar un archivo PDF consolidado que contenga la telemetría y los últimos resultados de todos los agentes conectados cuando se active el flujo de exportación de HTML a binario en n8n.

3.2 Requerimientos No Funcionales

Los requerimientos no funcionales se estructuran bajo las siguientes dimensiones de calidad de software:

3.2.1. Eficiencia de desempeño

- **RNF-ED-01. Tiempo de almacenamiento:** El sistema deberá registrar el comando enviado en la base de datos en menos de 5 segundos bajo condiciones normales de tráfico de red.
- **RNF-ED-02. Procesamiento de tareas:** El sistema deberá ejecutar la instrucción remota y enviar el resultado en menos de 5 segundos bajo la recepción de un comando simple de terminal.

3.2.2. Compatibilidad

- **RNF-COM-01. Sincronización API Rest:** El sistema deberá realizar el intercambio de datos entre n8n y Supabase en menos de 3 segundos bajo el uso de la interfaz web estándar.
- **RNF-COM-02. Coexistencia de procesos:** El sistema deberá inicializar el agente en segundo plano compartiendo recursos con procesos nativos en menos de 1 segundo bajo entornos operativos con alta demanda de CPU.

3.2.3. Usabilidad

- **RNF-USA-01. Despliegue de interfaz:** El sistema deberá cargar completamente el formulario web del operador en menos de 3 segundos bajo conexiones estándar de internet.
- **RNF-USA-02. Confirmación visual:** El sistema deberá mostrar el mensaje de éxito en pantalla en menos de 1 segundo bajo el envío exitoso de los datos del formulario.

3.2.4. Fiabilidad

- **RNF-FIA-01. Reconexión automática:** El sistema deberá reestablecer el enlace con la base de datos en la nube en menos de 10 segundos bajo un escenario de caída temporal de red.
- **RNF-FIA-02. Reporte de fallos:** El sistema deberá registrar los errores de ejecución en la base de datos en menos de 4 segundos bajo la introducción de sintaxis inválida por el operador.

3.2.5. Seguridad

- **RNF-SEG-01. Validation de tokens:** El sistema deberá verificar la clave secreta de Supabase en menos de 1 segundo bajo cada petición HTTP entrante desde el cliente.

3.2.6. Portabilidad

- **RNF-POR-02. Despliegue en el contenedor:** El sistema deberá instanciar el servicio local de n8n en menos de 15 segundos bajo una infraestructura de Docker estándar.

4 Funcionamiento Operacional

El funcionamiento de este MVP se basa en una arquitectura asíncrona y desacoplada de tres capas que permite la administración remota sin una conexión directa entre el operador y el objetivo.

El ciclo comienza en la interfaz de n8n, donde el operador utiliza un formulario web para ingresar un comando. En lugar de comunicarse directamente con la máquina de destino, n8n actúa como intermediario y realiza una petición API para registrar la instrucción de forma segura en la tabla de tareas de una base de datos centralizada en la nube mediante Supabase, quedando el comando en estado pendiente.

Por otro lado, el agente desarrollado en Python se ejecuta de forma persistente en la máquina remota utilizando una estrategia de consultas periódicas (*polling*). Cada pocos

segundos, el agente envía una señal de vida o *heartbeat* para actualizar su estado de disponibilidad en la base de datos y, simultáneamente, revisa si existen tareas asignadas a su identificador único.

Al detectar una orden pendiente, el script la extrae de Supabase y la ejecuta localmente a nivel de sistema operativo utilizando subprocesos internos de forma controlada.

Finalmente, una vez que el agente procesa la instrucción, captura el resultado de la terminal (*output*) y realiza una actualización en Supabase, cambiando el estado de la tarea a completado y almacenando la respuesta. Esto permite que el operador visualice el desenlace de la orden en tiempo real desde el panel de control.

Además, el flujo de n8n consolida de forma automática el historial de estas interacciones, transformando los datos recopilados en un reporte ejecutivo en formato PDF listo para su descarga y análisis.

5 Repositorio del Proyecto

El código fuente completo de los componentes del operador, configuraciones de la base de datos y el script del agente ejecutor en Python se encuentra alojado de forma pública para su revisión técnica en el siguiente repositorio:

<https://github.com/TatanPerez/Sarabhel>